

Vector Databases: ChromaDB

Vector Databases Versus Traditional Databases

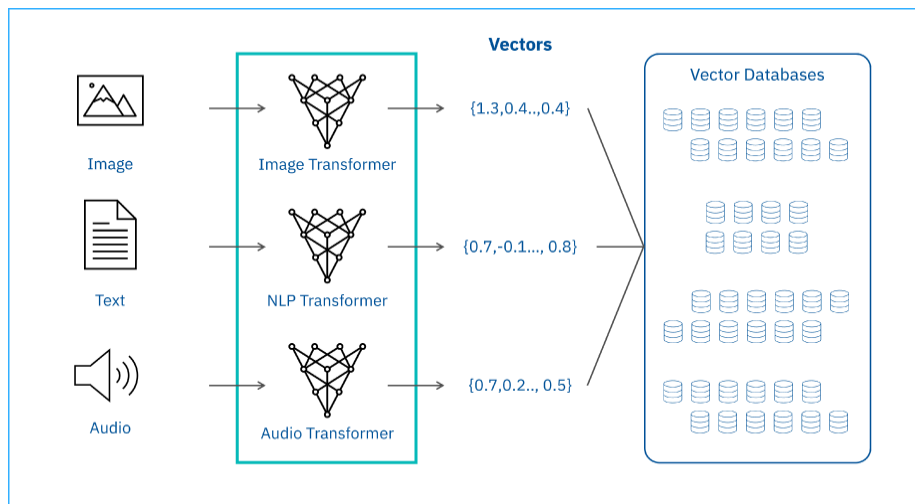
Vector databases

A vector database is a specialized database designed to store and query vectorized data rapidly. Unlike conventional databases, which organize data in tables, a vector database represents data as vectors in a multi-dimensional space. These vectors encapsulate essential attributes of the items they represent, making vector databases ideal for tasks requiring similarity searches, nearest neighbor queries, and assessing distances or similarities between vectors.

Vector databases data storage

Vector databases store data formatted so that numerical vectors depict each data item. These numbers signify various attributes or features of the object, with each vector dimension corresponding to a specific attribute. For instance, in an image database, each image may be represented as a vector of pixel values, while a text database might represent each piece of text as a vector of word frequencies.

In the following illustration you can see that you can take images, text (including text strings), and audio, then apply the appropriate transformer to create vector embeddings, and then store the data in vector databases for further use.



Vector libraries

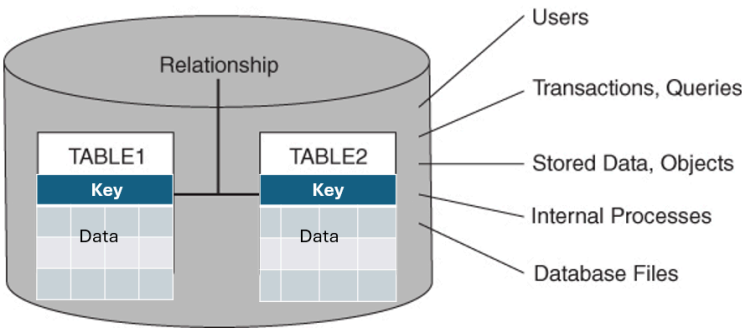
Vector libraries within in-memory vector databases. Besides libraries, Vector databases use pre-configured algorithms to store and update data. You'll find commercially available vector libraries for similarity. In contrast, vector databases have full CRUD (create, read, update, and delete) capabilities. Additionally, vector databases are often a part of enterprise-level production deployments.

Relational database organization

A relational database organizes data into tables, utilizing rows and columns. Relational databases use structured query language (SQL) for data querying and manipulation, adhering to the relational model. Relational databases excel in managing structured data where relationships between different entities are well-defined.

Relational database data storage

Relational databases store data in tables, each representing a distinct entity or relationship. Each row in a table corresponds to a record, while each column represents a property or attribute. Tables contain data and get connected, having relationships. with other tables using keys, such as main and foreign keys. In a relational database, users can perform transactions, queries, and internal processes on stored data or objects (database files) within rows and columns of tables



The database structures data in these tables and establishes connections between the tables using keys identified as primary and foreign keys. A relational database query manipulates the rows and columns using SELECT, INSERT, UPDATE, and DELETE operations.

Vector and relational databases compared

Function	Traditional databases	Vector databases
Data Representation	They organize data in a structured format using tables, rows, and columns, ideal for relational data.	They represent data as multi-dimensional vectors, efficiently encoding complex and unstructured data
Data Search and Retrieval	SQL queries are suited for traditional databases with structured data.	They specialize in similarity searches and retrieving vectorized data, facilitating tasks like image retrieval, recommendation systems, and anomaly detection.
Indexing	Traditional databases employ indexing methods like B-trees for efficient data retrieval.	They use indexing structures like metric trees and hashing suited for high-dimensional spaces, enhancing nearest-neighbor searches and similarity assessments.
Scalability	Scaling traditional databases can be challenging, often requiring resource augmentation or data sharding.	They are designed for scalability, especially in handling large datasets and similarity searches, using distributed architectures for horizontal scaling.
Applications	They are pivotal in business applications and transactional systems where structured data is processed.	They shine in analyzing vast datasets, supporting fields like scientific research, natural language processing, and multimedia analysis.

Types of Vector Databases

In-memory

- Fast read/write (good for real-time use)
- Examples: RedisAI, Torchserve

Disk-based

- Suitable for large datasets
- Use indexing and compression
- Examples: Annoy, Milvus, ScaNN

Distributed

- Spread data across nodes, scalable & fault-tolerant
- Examples: FAISS, Elasticsearch (with plugin), Dask-ML

Graph-based

- Represent vectors as graph attributes (nodes & edges)
- Good for complex relationships
- Examples: Neo4j, Amazon Neptune, TigerGraph

Time-series

- For vector data over time (e.g., IoT, anomaly detection)
- Examples: InfluxDB, TimescaleDB, Prometheus

Dedicated Vector Databases vs Databases with Vector Support

Dedicated Vector Databases

- Built specifically for vector operations
- Use optimized data structures (LSH, quantization, etc.)
- Fast, scalable, and highly customizable
- Examples: FAISS, Annoy, Milvus

Databases that Support Vector Search

- Traditional DBs with vector features/plugins
- Not as optimized but more versatile
- Store vectors as arrays, BLOBs, UDTs
- Examples: PostgreSQL (PostGIS), MySQL, RedisAI, Elasticsearch, MongoDB, Cassandra, SingleStore

Applications of Vector Databases

Image and Video Analysis

- Store high-dimensional feature vectors (e.g., color, texture, deep embeddings).
- Enable similarity searches to find or recommend similar media.
- Power real-time tasks like surveillance, object detection, and live video analysis.

Recommendation Systems

- Use embeddings and nearest neighbor search for personalized recommendations.
- Support cross-domain suggestions (e.g., movies, products).
- Scalable to handle many users with fast indexing and retrieval.

Geospatial Analysis & Location-Based Services

- Use R-trees, quadtrees for spatial indexing (e.g., GPS, polygons).
- Enable range queries, proximity searches, and real-time location analytics.
- Applications: navigation apps, fleet management, event suggestions.

Marketing and Social Insights

- Handle large-scale user data for trends, interests, and behavior tracking.
- Provide real-time analytics with caching and parallel processing.
- Enable autoscaling to adjust resources based on demand (e.g., growing user base).

Key Takeaways

- Vector databases support real-time, scalable, and intelligent applications.
- They power media analysis, recommendation engines, location-aware apps, and marketing platforms.
- Their use of embeddings, spatial indexing, and horizontal scalability make them vital for modern data-driven systems.

Chroma DB

- Chroma DB is a specialized vector database known for its ease of configuration and customization and its operational efficiencies for structured and unstructured data.

Data storage and organization in Chroma DB

- Chroma DB stores data objects using embeddings. These data objects exist within the database in logical, related groups or containers known as collections.

When creating a collection for storing data, you might decide to group a related set of data objects in a collection based on one major dimensional characteristic. For example, you create several movie collections using the categories of comedy, action and adventure, documentaries, and children's movies. Each movie within a collection is related to one major characteristic and might share another minor characteristic, such as being set in New York City.

However, the person using your database to locate movies might want to do a similarity search to see movies of all types set in New York City. Chroma DB is not limited to queries within one collection. The requestor's query will access multiple movie "collections" within Chroma DB and list all movies set in New York City.

Chroma DB advantages

- Enhanced data relevance: Vector embeddings, using dimensions and other Chroma data storage characteristics, support concise searches for more precise results.

- Improved data management: Chroma DB supports code that speeds data addition, updates, and deletion. You can even use Chroma DB to list collections based on a specific code.

Why Docker with Chroma DB

- Monolithic applications are outdated. You can use your Chroma database and Docker's containerization capabilities with a single, one-line command. For example, you can initiate a Docker container containing the Chroma DB image using the following command.
`docker run -p 8000:8000 chromadb/chroma`

Note: To run your Chroma DB instance on localhost, you must install Docker based on your operating system configuration.

How Chroma DB Helps LLMs

Chroma is a vector database that:

- Stores objects (like "spoon" or "pen") as vectors with features across multiple dimensions (e.g., function, shape).
- Uses vector comparisons to clarify differences and similarities.

Example: Even though both a spoon and pen are "holdable," their function dimensions differ. Chroma helps LLMs recognize this, preventing faulty conclusions (e.g., "you can write with a spoon").

How Chroma Addresses LLM Limitations:

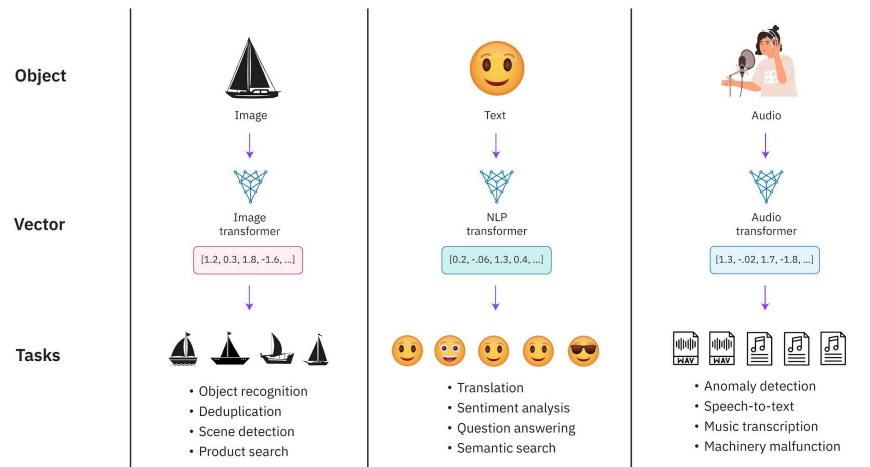
- Reduces bias by using diverse data sources
- Improves accuracy by integrating factual knowledge
- Supports basic understanding via encoded relationships (common-sense logic)

Chroma DB Architecture

- Obtain Embeddings – Convert data (text, image, etc.) to vectors
- Create Collections – Like tables in SQL; logical grouping
- Store Information – Save vectors and associated metadata
- Collection Operations – Delete, update, rename collections
- Query & Group – Use text or vector queries for semantic search

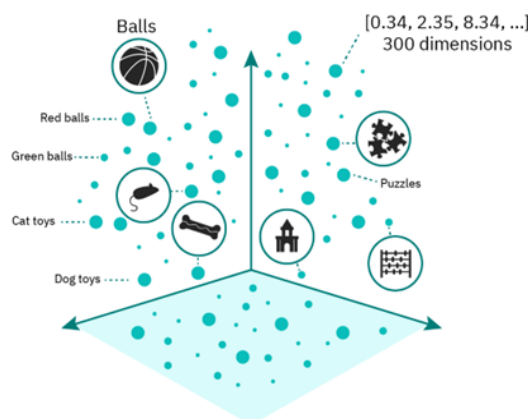
Vector Embeddings and Vector Databases

Traditional databases are great at storing and retrieving data based on exact matches. However, the world of AI necessitates a more nuanced approach to evaluate data based on similar characteristics. Vector embeddings are essential for this data task. Imagine representing data points, including text, images, and other complex data types, as vectors in a high-dimensional space. Vector points that exist closer together in this space hold greater semantic similarity. As seen in the following infographic, you can analyze vectorized images for tasks such as object recognition, deduplication, scene detection, and product search. You can analyze text for translation, sentiment analysis, answers to questions, and semantic search. And finally, you can analyze audio files for anomaly detection, speech-to-text, music transcripts, and machinery malfunctions. Summarized, vector data transformation is available for images, text and audio data analysis.



Next, imagine you have a room with a bunch of different toys scattered around. Each toy is represented by a list of qualities, like color, shape, size, and material. Now, imagine each toy as a point in a space where each quality is in a different direction; you'd have a high-dimensional space. Similar toys, like two different balls, would be closer together in this space because they share many qualities, while something different, like a doll, might be farther away. So, when representing data points as vectors in a high-dimensional space, it's like assigning each toy a unique set of coordinates based on its qualities, and toys with similar coordinates (or qualities) end up closer together.

A Chroma DB vector database could be incredibly useful in the scenario described above. Let's say you have a collection of images of toys, and you want to organize them based on their visual similarities. You could use Chroma DB to convert each image into a high-dimensional vector representation, where each dimension represents different aspects of the image, such as color distribution, texture, and shape. Once you have these vector representations, you can store them in the Chroma DB database. Now, when you want to find similar images to a given one, you can simply query the database with the vector representation of that image. Chroma DB will then efficiently search through all the vectors in the database and retrieve the ones that are closest to the query vector.



This way, you can quickly find images of toys that are visually similar to a specific toy without manually comparing each image against the entire collection. This can be incredibly helpful for tasks like image recommendation, visual search, and content-based image retrieval, making organizing and exploring large collections of visual data easier.

Unveiling the Vector Data Architecture of Chroma DB

Chroma DB boasts a well-defined architecture that is meticulously crafted for speed and scalability. Let's examine its key components:

- **Vector Embeddings:** Chroma DB integrates seamlessly with various embedding models. By default, it utilizes Sentence Transformers to convert text data into vector embeddings. However, you can employ other models like OpenAI embeddings or even introduce your own custom models.

Vector Embedding options for text

- **TensorFlow embeddings:** TensorFlow offers a powerful and versatile framework for building and deploying machine learning models and provides tools for creating and manipulating vector embeddings. You can leverage pre-trained models available in TensorFlow Hub, which encompasses a wide range of tasks and languages. You can fine-tune these models, trained on massive datasets, for your specific domain. Alternatively, you can build your own custom embedding model using TensorFlow's machine-learning libraries. This approach offers maximum control but requires expertise in model development and data preparation.
- **OpenAI embeddings:** OpenAI, a research company focused on artificial intelligence, has developed several powerful large language models, including GPT-3. OpenAI offers access to pre-trained embeddings derived from these models. These embeddings are known for their ability to capture complex semantic relationships between words.

It's important to note that access to OpenAI's embeddings might be limited or require specific permissions. Additionally, compared to TensorFlow Hub, fewer pre-trained options might be readily available.

Choosing the Right Embedding Model

The best choice depends on your specific needs and data. Consider these factors:

- **Domain specificity:** If your data has a particular domain, such as legal documents or medical records), pre-trained models optimized for that domain might be available in TensorFlow Hub.
- **Control compared to ease of use:** Pre-trained models offer a quick and easy solution, while custom models allow for fine-tuning but require more development effort.
- **Access and cost:** OpenAI embeddings might have limitations on access or require specific agreements or monetary compensation.

Analyzing the data: Similarity search algorithms

- **The nearest neighbor search algorithm:**

What does the nearest neighbor search algorithm indicate about the point of reference or the query?

Depending on the distance or the similarity metric, the nearest neighbor indicates that most features of the query are similar to that of the nearest neighbor. You can also use this algorithm to synthesize data. For instance, Nearest neighbor algorithm is used in image data for pixelating and improving the quality of the image. This algorithm represents complex data such as text, images, videos, and others as vectors in a highly multidimensional space, considering various data attributes. For instance, text data attributes could include vectors that represent language, tense, tone, and sentiment. During the machine learning process, developers create and train a model on all the data, considering specific attributes. The training on these

features generates the vector output. Since the training involves the same attributes, vectors that are closest to each other or have the highest similarity scores represent similar data. How far can the nearest neighbor be? There should be a threshold that needs to be set to consider the validity of the nearest neighbor. If there are datapoints in the set, one of the datapoints in all probabilities will be closer than the others, making it the nearest neighbor in comparison. But that doesn't have to necessarily indicate that there is any sort of similarity between the two data points. It is always logical to set a threshold or distance or similarity to consider a point as the nearest neighbor. For example, if the cosine similarity between two vectors is .5, it makes sense to infer that there are reasonable common features. But if the similarity is closer to 0, but a valid number, then similarity is minimal.

The Chroma DB query engine

The heart of Chroma DB lies in its powerful query engine. This engine is optimized for nearest neighbor search methods. Instead of searching for exact matches, the database can apply the techniques needed to identify data points with the closest vectors in the embedding space, effectively returning the most semantically similar information.

Chroma DB empowers developers with user-friendly Software Development Kits (SDKs). Currently, Chroma offers Python and JavaScript/TypeScript SDKs, which streamline the process of interacting with the database from this application code.

Additional query types available using Chroma DB

- Range search:
 - This query retrieves embeddings within a specific radius of the query vector. It allows you to find data points that share a certain level of semantic similarity with your query but not necessarily the absolute closest ones.
 - Example: You might use a range search to recommend similar movies to a user based on a movie they enjoyed. The retrieved embeddings wouldn't necessarily be the most similar movies, but they would share some level of thematic or genre similarity.
- Filtering by metadata:
 - While Chroma DB excels at semantic similarity search, you can combine vector searches with filtering based on additional metadata associated with the embeddings.
 - Example: You could search for similar products (using nearest neighbor search) and then filter the results only to show products within a specific price range.

Implementing Chroma DB: A summarized step-by-step process

1. Get embeddings: Take the actions needed to convert words, images, or data into numbers and, specifically, vector representations.
2. Create collections: Create collections to group data like tables in a relational database.
3. Put data into collections: Save the data, like text and embeddings, in collections after preprocessing the data and generating vector embeddings.
4. Perform collection operations: Use Chroma DB to delete, change, or rename collections, among other actions, which gives you more ways to work with your data.
5. Use text or vector searches to find information to locate your answers based on semantic meaning or vector similarity, which is a useful way to get the most out of your data.

Chroma DB: Creating Collections and Vector Embeddings

Collections in Chroma DB

- Act like relational database tables.
- Containers that store:
- Vector embeddings
- Documents (text, images, audio, video)
- Associated metadata (labels, tags)
- Enable categorized grouping of data for improved retrieval and management.

Vector Embeddings

- Fixed-length numerical vectors representing data meaning in high-dimensional space.
- Capture relationships and similarities between data points.
- Example: Text like "dog" represented as a vector with multiple dimensions (50-300 numbers).
- Similarity scores indicate closeness of vectors (e.g., "dog" is close to "puppy," "canine").

Steps to Create Collections and Embeddings in Chroma DB (JavaScript example)

- Import Chroma DB library components and initialize client connection.
- Define a default embedding function (DefaultEmbeddingFunction) to convert text to vectors.
- Create a collection using `client.createCollection`, specifying the collection name and embedding function.
- Generate embeddings for text data with `default_ef.generate`.
- Add documents and their embeddings to the collection with `collection.add`.
- Log embeddings to verify data added correctly.

Similarity Searches Using Chroma DB

Purpose of Similarity Search

- Fundamental in information retrieval and data analysis.
- Finds items in a dataset similar to a given query item.
- Similarity depends on context, data type, and application domain.

Five Types of Similarity Searches

- Text Document Similarity: Compares documents using vector embeddings (word/document embeddings) for semantic or syntactic similarity.
- Image/Video Similarity: Finds similar images/videos by comparing features like color histograms and neural network embeddings.
- Structured Data Similarity: Matches records in databases or tables using metrics like Euclidean distance (numerical data) or edit distance (text).
- Recommendation Systems: Suggests items (movies, books) based on user interaction history and similarity to previously liked items.
- Genomics/Bioinformatics: Searches for similar DNA, RNA, or protein sequences.

Essential vector operations for similarity search

- **Data Conversion to Vectors:** Convert data in different forms, such as text descriptions, images, or product details, into numerical vectors. Word embedding can be used for text, and feature extraction for images. This conversion mathematically captures the essence of the data.
- **Generating Similarity Metrics:** Define distance metrics, like cosine similarity or Euclidean distance, to determine how close two vectors are in high-dimensional space. The closer the vectors are, the more similar the data points are.
- **Creating a Query as a Vector:** Transform the user query into a vector using the same transformation used on the data. This way, the query can be compared directly to the data vectors.
- **Searching for Similar Data:** It searches for documents in a collection that have vector representations that are most similar to the query vector. This process locates relevant data points to the user's search intent.
- **Retrieval and Presentation of the Results:** Once it finds relevant data points, it fetches the relevant documents and presents them to the user, along with additional metadata (like categories and similarity scores) to enhance user comprehension and satisfaction.

Chroma DB architecture progressively applies the following phases: obtain embeddings, create collections, store information within the collections, apply collection operations, and query and group data.

Similarity search is a fundamental operation in information retrieval and data analysis, used to find items in a data set similar to a given reference or queried item.

Text documents, images and video, structured data, recommendations, and genomics and bio information, implement unique code for each type of similarity search.

For essential operations using Chroma DB, you use the keyword `async` to wait for tasks to complete, the `try...catch` block to handle potential errors, the `console.log(...)` to log the successful task completion, and the `console.error(...)` to log error messages

To create a collection and embeddings in Chroma DB, you create code that imports the ChromaDB components, sets up the collection and embedding functions, creates your collection, and generates your embeddings.